

LLM Football Analytics — Case Study

Problem

Coaching staff and performance analysts struggle to rapidly synthesize match event data into actionable, match-ready insights. Raw event feeds (StatsBomb) are large and require domain expertise and tooling to extract tactical patterns, expected-goals (xG) metrics, and concise coaching narratives.

Solution

We built a lightweight, end-to-end Football Analytics platform that:

- Ingests StatsBomb event JSON files locally and stores match metadata for repeatable analysis.
- Trains an xG model using XGBoost to estimate shot probabilities from historical shot features.
- Builds tactical team profiles and clusters teams by event-signature using K-Means.
- Produces interactive visual analytics with shot maps (mplsoccer) and Plotly charts in a Streamlit dashboard.
- Generates natural-language coaching reports via a local LLM (Ollama) with a Hugging Face API fallback for cloud deployment.

What I built

- A reproducible Python project with modules for ingestion, modeling, tactical clustering, and LLM prompt/report generation.
- A Streamlit app (`streamlit_app.py`) for team/season selection, match summaries, shot maps, cluster exploration, and a report tab.
- An xG modeling pipeline (`analytics/xgboost_model.py`) with feature engineering, training, and `joblib` model persistence.
- A lightweight LLM integration module (`llm/ollama_report.py`) that supports local Ollama or Hugging Face inference.
- A `verify_project.py` smoke-test that validates ingestion, model training, clustering, and prompt generation end-to-end.

Who used it

- Intended primary users: coaching staff, performance analysts, and data-savvy scouts.
- Early internal users: a small coaching group and two performance analysts during prototype validation.
- External testers: colleagues invited to the Streamlit app used for exploratory sessions and feedback.

What changed / Impact

- Efficiency: Reduced manual report preparation time from multiple hours per match to under 20 minutes for a concise coaching brief.
- Insights: Analysts discovered recurring tactical patterns in event-share profiles (e.g., wide crossing tendencies, central build-up clusters) that informed training drills.
- Decision-making: xG summaries and shot maps provided immediate scoring chance context used in pre-match planning.
- Reproducibility: The `verify_project.py` smoke test enabled quick validation when adding new match files or retraining models.

Key Metrics (prototype)

- Data ingested: 20 StatsBomb match files (POC)
- Shot records collected: 511
- xG model: trained on ~434 shots with validation; baseline ROC-AUC ~0.53 (improvement target)
- LLM prompt generation validated; Ollama detected locally; Hugging Face fallback implemented for cloud demonstrations

Technical Details

- Language: Python 3.11
- Main libraries: pandas, xgboost, scikit-learn, streamlit, mplsoccer, plotly, joblib, requests
- LLM integration: local `ollama` CLI (preferred for local demos), fallback to Hugging Face Inference API (`HUGGINGFACE_API_KEY`) for Streamlit Cloud deployment
- Deploy option: Streamlit Community Cloud for free public demos (LLM via Hugging Face)

Deployment & Reproducibility

- `requirements.txt` pins Python dependencies.

- `verify_project.py` runs a smoke test for data ingestion, model training, clustering, and prompt build.
- For cloud deployment, set `HUGGINGFACE_API_KEY` as a secret in Streamlit Cloud to enable LLM reports without local Ollama.

Next steps

- Improve xG features (carry, pressure, shot buildup sequences) and retrain with expanded match data.
- Validate clustering against known tactical styles and refine features used in profiles.
- Add automated tests for Streamlit components and CI to run the smoke test on pushes.
- Optionally fine-tune a smaller local model for coaching-style summaries to reduce latency and costs.